

8. 선형대수 방정식과 그 해법

Linear Algebraic Equations and Solution Methods

Jin Hee Yoon

Dept. of Artificial Intelligence and Information Technology

Sejong University

Spring, 2026

이번 강의에서는 선형대수 방정식의 수학적 구조와 MATLAB을 이용한 해법을 함께 학습한다.

- 선형시스템 $Ax = b$ 의 의미와 해의 존재 조건을 이해한다.
- 역행렬, 행렬 나눗셈, rank, determinant를 이용한 해석 방법을 학습한다.
- overdetermined system과 least squares solution의 의미를 이해한다.
- ill-conditioned system에서 수치해의 불안정성을 확인한다.
- MATLAB을 이용하여 실제 데이터 기반 선형모델을 구성하고 해를 계산한다.

선형대수 방정식의 기본 형태

선형대수 방정식은 여러 개의 미지수가 선형적으로 연결된 방정식들의 모음이다. 가장 일반적인 형태는 다음과 같다.

$$Ax = b$$

여기서

- A : 계수행렬(coefficient matrix)
- x : 미지수 벡터(unknown vector)
- b : 상수항 벡터(right-hand side vector)

해석

선형시스템을 푼다는 것은 A 와 b 가 주어졌을 때, 방정식 $Ax = b$ 를 만족하는 벡터 x 를 찾는 것이다.

2변수 선형시스템의 기하학적 의미

2개의 미지수 x, y 를 갖는 선형시스템은 평면 위의 직선들의 교점을 찾는 문제로 해석할 수 있다.

$$\begin{cases} a_{11}x + a_{12}y = b_1 \\ a_{21}x + a_{22}y = b_2 \end{cases}$$

- 두 직선이 한 점에서 만나면 해가 하나 존재한다.
- 두 직선이 평행하면 해가 존재하지 않는다.
- 두 직선이 완전히 같으면 무한히 많은 해가 존재한다.

중요한 관점

선형대수 방정식의 해는 단순한 계산 결과가 아니라, 행렬이 만드는 기하학적 구조와 직접 연결된다.

예제 1: 간단한 2×2 선형시스템

Example

다음 선형시스템을 MATLAB으로 풀어보자.

$$\begin{cases} 2x + 3y = 8 \\ 4x - y = 2 \end{cases}$$

이 문제는 두 직선의 교점을 찾는 문제이며, 행렬 형태로는 $Ax = b$ 로 표현된다.

```
clear; clc;                                % Clear workspace

A = [2 3; 4 -1];                            % Coefficient matrix
b = [8; 2];                                  % Right-hand side vector

x = A\b;                                     % Solve Ax=b

disp(x);                                     % Display solution
```

MATLAB에서 선형시스템을 푸는 대표 방법

MATLAB에서 $Ax = b$ 를 푸는 대표적인 방법은 다음과 같다.

방법	MATLAB 코드	특징
역행렬 사용	<code>x = inv(A)*b</code>	개념적으로 직관적
행렬 나눗셈	<code>x = A\b</code>	수치적으로 안정적
rank 분석	<code>rank(A)</code>	해 존재성 분석
determinant	<code>det(A)</code>	정사각 행렬의 가역성 확인

수업에서 강조할 점

실제 계산에서는 `inv(A)*b`보다 `A\b`를 사용하는 것이 일반적으로 더 안정적이다. `A\b`는 역행렬을 구하지 않고 Gauss-Jordan 소거법, LU 분해 및 Pivoting을 사용한다.

예제 2: $\text{inv}(A)*b$ 와 $A \setminus b$ 비교

Example

같은 선형시스템을 역행렬 방법과 행렬 나눗셈 방법으로 각각 풀고 결과를 비교한다. 작은 문제에서는 결과가 거의 같지만, 큰 문제나 불안정한 행렬에서는 차이가 발생할 수 있다.

```
clear; clc;                                % Clear workspace

A = [5 2 -1; -2 6 3; 1 -1 7];             % Coefficient matrix
b = [8; 5; 12];                           % Right-hand side vector

x1 = inv(A)*b;                             % Solution using inverse
x2 = A\b;                                  % Solution using backslash

diff_x = x1 - x2;                          % Difference between solutions

disp(x1);                                  % Display inverse solution
disp(x2);                                  % Display backslash solution
disp(diff_x);                              % Display difference
```

해의 존재성과 rank

선형시스템 $Ax = b$ 의 해 존재 여부는 rank를 이용하여 분석할 수 있다.

$$\text{rank}(A) = \text{rank}([A \ b])$$

이면 해가 존재한다.

$$\text{rank}(A) < \text{rank}([A \ b])$$

이면 해가 존재하지 않는다.

해의 개수

- $\text{rank}(A) = \text{rank}([A \ b]) = n$: 유일해
- $\text{rank}(A) = \text{rank}([A \ b]) < n$: 무한히 많은 해
- $\text{rank}(A) < \text{rank}([A \ b])$: 해 없음

예제 3: rank를 이용한 해 존재성 확인

Example

다음 선형시스템에 대해 계수행렬 A 와 첨가행렬 $[A \ b]$ 의 rank를 비교한다. rank 비교를 통해 해가 존재하는지, 유일한지, 무한히 많은지 판단할 수 있다.

```
clear; clc;                                % Clear workspace

A = [1 2 3; 2 4 6; 1 1 1];                % Coefficient matrix
b = [6; 12; 3];                             % Right-hand side vector

rank_A = rank(A);                           % Rank of A
rank_aug = rank([A b]);                     % Rank of augmented matrix
n = size(A,2);                              % Number of unknowns

fprintf('rank(A) = %d\n', rank_A);
fprintf('rank([A b]) = %d\n', rank_aug);
fprintf('number of unknowns = %d\n', n);
```

determinant와 가역성

정사각 행렬 A 에 대해 determinant는 가역성(invertibility)을 판단하는 데 사용된다.

$$\det(A) \neq 0 \Rightarrow A^{-1} \text{ exists}$$

따라서 A 가 정사각 행렬이고 $\det(A) \neq 0$ 이면 $Ax = b$ 는 모든 b 에 대해 유일해를 가진다.

주의

$\det(A)$ 는 작은 행렬에서는 유용하지만, 큰 행렬이나 수치적으로 불안정한 문제에서는 rank, condition number, backslash 사용이 더 실용적이다.

예제 4: determinant와 해 계산

Example

계수행렬의 determinant를 계산하여 행렬이 가역인지 확인하고, 가역이면 선형시스템의 해를 계산한다.

```
clear; clc;                                % Clear workspace

A = [3 -1 2; 1 4 -2; 2 1 5];              % Coefficient matrix
b = [10; -1; 7];                           % Right-hand side vector

d = det(A);                                % Determinant of A

fprintf('det(A) = %.4f\n', d);

if d ~= 0
    x = A\b;                                % Solve if A is invertible
    disp(x);
else
    disp('A is singular.');
```

end

Residual의 의미

계산된 해 $\hat{x}$ 가 실제로 $Ax = b$ 를 얼마나 잘 만족하는지 확인하려면 residual을 계산한다.

$$r = b - A\hat{x}$$

$$\|r\| = \|b - A\hat{x}\|$$

- residual이 0에 가까우면 방정식을 잘 만족한다.
- residual이 크면 해가 부정확하거나 시스템이 일관되지 않을 수 있다.

실무적 의미

선형시스템을 풀 때 해 자체뿐 아니라 residual norm도 함께 확인하는 것이 중요하다.

예제 5: residual 계산

Example

선형시스템 $Ax = b$ 를 푼 뒤, residual vector와 residual norm을 계산한다. 이를 통해 계산된 해가 원래 방정식을 얼마나 잘 만족하는지 확인한다.

```
clear; clc;                                % Clear workspace

A = [4 1 -1; 2 7 1; -1 3 12];             % Coefficient matrix
b = [3; 19; 31];                           % Right-hand side vector

x = A\b;                                    % Solve Ax=b

r = b - A*x;                               % Residual vector
res_norm = norm(r);                         % Residual norm

disp(x);                                    % Display solution
disp(r);                                    % Display residual vector
fprintf('Residual norm = %.4e\n', res_norm);
```

Overdetermined System

방정식 개수가 미지수 개수보다 많은 경우를 overdetermined system 이라고 한다.

$$A \in \mathbb{R}^{m \times n}, \quad m > n$$

이 경우 보통 모든 방정식을 정확히 만족하는 해가 존재하지 않는다. 따라서 다음 least squares problem을 푼다.

$$\min_x \|Ax - b\|^2$$

해석

least squares solution은 모든 방정식을 완벽하게 만족하는 해가 아니라, 전체 오차의 제곱합을 가장 작게 만드는 해이다.

예제 6: Least Squares 직선 피팅

Example

측정 데이터 (t, y) 가 주어졌을 때, $y \approx c_1 + c_2 t$ 형태의 직선을 least squares로 찾는다. 행렬 $A = [1 \ t]$ 를 구성하고 $A \backslash y$ 를 이용하여 계수를 계산한다.

```
clear; clc; close all;           % Clear workspace

t = [0; 1; 2; 3; 4; 5];         % Input data
y = [2.1; 2.9; 4.2; 5.1; 5.8; 7.2]; % Observed output
A = [ones(size(t)) t];          % Design matrix
c = A \ y;                       % Least squares solution
y_fit = A * c;                   % Fitted values

plot(t, y, 'o', 'LineWidth', 1.5); % Plot data
hold on;
plot(t, y_fit, '-', 'LineWidth', 2); % Plot fitted line
legend('Data', 'Least squares line');
grid on;
```

Normal Equation

least squares problem

$$\min_x \|Ax - b\|^2$$

의 해는 다음 normal equation을 만족한다.

$$A^T Ax = A^T b$$

따라서

$$x = (A^T A)^{-1} A^T b$$

로 표현할 수 있다.

주의

normal equation은 수학적으로 중요하지만, 실제 MATLAB 계산에서는 $A \setminus b$ 를 사용하는 것이 더 안정적이다.

예제 7: Normal Equation과 Backslash 비교

Example

같은 least squares 문제를 normal equation과 $A \backslash b$ 로 각각 풀어본다. 두 결과는 비슷하지만 수치적으로는 $A \backslash b$ 방식이 더 안정적이다.

```
clear; clc;                                % Clear workspace

t = (0:0.5:5)';                             % Input data
y = 1.5 + 2.2*t + 0.2*randn(size(t)); % Noisy linear data

A = [ones(size(t)) t];                     % Design matrix

c_backslash = A \ y;                        % Backslash solution
c_normal = inv(A'*A)*(A'*y);                % Normal equation solution

disp(c_backslash);
disp(c_normal);
disp(c_backslash - c_normal);
```

Condition Number

condition number는 선형시스템이 수치적으로 얼마나 민감한지 나타낸다.

$$\kappa(A) = \|A\| \|A^{-1}\|$$

- $\kappa(A)$ 가 작으면 안정적인 문제이다.
- $\kappa(A)$ 가 크면 작은 입력 오차가 큰 해의 오차로 증폭될 수 있다.

수치해석 관점

해가 존재하더라도 condition number가 크면 계산된 해를 신뢰하기 어려울 수 있다.

Condition Number의 해석

Condition Number의 크기에 따라 행렬의 수치적 안정성을 판단할 수 있다.

$$\kappa(A) \approx 1$$

Well-Conditioned Matrix

입력 데이터에 작은 오차가 발생하더라도 해의 변화가 크지 않다. 수치적으로 안정적이며 계산 결과를 신뢰할 수 있다.

$$\kappa(A) \gg 1$$

Ill-Conditioned Matrix

입력 데이터의 아주 작은 변화도 해에서 크게 증폭될 수 있다. 즉, b 가 조금만 변해도 해 x 가 크게 달라질 수 있으며, 수치적으로 불안정한 행렬이다.

예제 8: ill-conditioned system

Example

Hilbert matrix는 대표적인 ill-conditioned matrix이다. 작은 크기에서도 condition number가 매우 커질 수 있으며, 작은 b 의 변화가 해에 큰 영향을 줄 수 있다.

```
clear; clc;                                % Clear workspace

n = 8;                                     % Matrix size
A = hilb(n);                              % Hilbert matrix
x_true = ones(n,1);                       % True solution
b = A*x_true;                             % Exact right-hand side
x_est = A\b;                              % Computed solution
cond_A = cond(A);                         % Condition number
err = norm(x_est - x_true);               % Error norm

fprintf('cond(A) = %.4e\n', cond_A);
fprintf('error norm = %.4e\n', err);
```

Sparse Linear System

대규모 선형시스템에서는 대부분의 원소가 0인 sparse matrix가 자주 등장한다.

$$Ax = b, \quad A \text{ is sparse}$$

- 메모리를 적게 사용한다.
- 큰 시스템을 더 효율적으로 계산할 수 있다.
- 과학계산, 회로해석, 네트워크 분석에서 자주 사용된다.

MATLAB 함수

`sparse`, `spdiags`, `speye` 등을 이용하여 sparse matrix를 만들 수 있다.

예제 9: Tridiagonal Sparse System

Example

1차원 격자 문제에서 자주 등장하는 tridiagonal matrix를 sparse matrix로 생성하고 선형시스템을 푼다.

```
clear; clc;                                % Clear workspace

n = 100;                                    % Size of system
e = ones(n,1);                             % Vector of ones
A = spdiags([-e 2*e -e], -1:1, n, n); % Tridiagonal matrix
b = ones(n,1);                             % Right-hand side vector
x = A\b;                                    % Solve sparse linear system

plot(x,'LineWidth',1.5);                   % Plot solution
xlabel('Index');
ylabel('Solution value');
grid on;
```

선형시스템과 실제 모델링

선형시스템은 다양한 실제 문제에 등장한다.

- 전기회로: Kirchhoff 법칙으로부터 선형시스템 구성
- 데이터 피팅: 관측 데이터를 설명하는 선형모델 추정
- 구조해석: 힘과 변위의 관계 계산
- Markov chain: 정상상태 확률 계산
- 이미지 처리: smoothing, reconstruction 문제

핵심

MATLAB에서 선형시스템을 푸는 것은 단순한 수식 계산이 아니라 실제 시스템을 수치적으로 분석하는 기본 도구이다.

예제 10: 회로 방정식 풀이

Example

다음 회로 방정식이 주어졌다고 하자. 미지수 i_1, i_2, i_3 는 각 회로 가지의 전류를 의미한다.

$$\begin{bmatrix} 10 & -2 & -1 \\ -2 & 8 & -3 \\ -1 & -3 & 6 \end{bmatrix} \begin{bmatrix} i_1 \\ i_2 \\ i_3 \end{bmatrix} = \begin{bmatrix} 12 \\ 5 \\ 7 \end{bmatrix}$$

```
clear; clc;                                % Clear workspace

A = [10 -2 -1; -2 8 -3; -1 -3 6];          % Circuit matrix
b = [12; 5; 7];                             % Voltage/source vector

I = A\b;                                     % Current solution

disp(I);                                    % Display currents

residual = norm(b - A*I);                   % Residual norm
fprintf('Residual norm = %.4e\n', residual);
```


실습문제 1-2

1. 고정밀 4×4 선형시스템 분석

다음 시스템을 MATLAB으로 풀고, determinant, rank, residual norm을 모두 계산하시오.

$$A = \begin{bmatrix} 8 & -2 & 1 & 0 \\ -2 & 7 & -3 & 1 \\ 1 & -3 & 9 & -2 \\ 0 & 1 & -2 & 6 \end{bmatrix}, \quad b = \begin{bmatrix} 10 \\ -4 \\ 15 \\ 3 \end{bmatrix}$$

- $A \backslash b$ 로 해를 구하시오.
- $\det(A)$, $\text{rank}(A)$, $\text{norm}(b - A \cdot x)$ 를 출력하시오.

2. 거의 특이한 행렬의 수치 불안정성

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 + \epsilon \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 2 + \epsilon \end{bmatrix}$$

에 대해 $\epsilon = 10^{-1}, 10^{-3}, 10^{-6}, 10^{-10}$ 일 때 해와 condition number를 비교하시오.

실습문제 3-4

3. 세 개의 데이터 센서를 이용한 선형 보정 모델

센서 입력 x 와 출력 y 가 다음 관계를 따른다고 가정한다.

$$y \approx c_1 + c_2x + c_3x^2$$

$x = 0 : 0.5 : 8$ 에 대해 $y = 2 + 1.5x - 0.2x^2 + \epsilon$, $\epsilon = 0.3\text{randn}$ 으로 데이터를 생성하시오.

- design matrix를 만들고 least squares 계수를 구하시오.
- 원 데이터와 fitted curve를 함께 그리시오.

4. inconsistent system 판별

다음 시스템의 rank를 분석하여 해가 존재하는지 판단하시오.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 6 \\ 13 \\ 3 \end{bmatrix}$$

실습문제 5-6

5. Overdetermined system에서 residual 분석

10개의 관측값을 이용하여 $y \approx a + bt$ 모델을 구성하시오.

$$t = 0, 1, \dots, 9$$

$$y = [1.2, 2.1, 2.9, 4.2, 5.1, 5.8, 7.0, 8.1, 8.9, 10.2]^T$$

- least squares 해를 구하시오.
- residual vector와 residual norm을 계산하시오.

6. Sparse tridiagonal system

$n = 200$ 에 대해 diagonal은 4, 양옆 diagonal은 -1인 sparse matrix를 만들고 $Ax = \mathbf{1}$ 을 푸시오. 해 x 를 그래프로 나타내시오.

실습문제 7-8

7. 정규방정식과 backslash 비교

다항식 모델

$$y \approx c_1 + c_2x + c_3x^2 + c_4x^3$$

을 noisy data에 대해 fitting하시오. $A \backslash y$ 와 $\text{inv}(A' * A) * (A' * y)$ 결과를 비교하시오.

8. 회로 전류 문제

다음 회로 방정식의 전류 벡터를 구하고 가장 큰 절댓값을 갖는 전류를 찾으시오.

$$\begin{bmatrix} 12 & -3 & 0 & -2 \\ -3 & 10 & -4 & 0 \\ 0 & -4 & 9 & -1 \\ -2 & 0 & -1 & 7 \end{bmatrix} i = \begin{bmatrix} 10 \\ 5 \\ 3 \\ 4 \end{bmatrix}$$

실습문제 9-10

9. Hilbert matrix 실험

$n = 4, 6, 8, 10, 12$ 에 대해 Hilbert matrix H_n 을 만들고, $x_{\text{true}} = \mathbf{1}$, $b = H_n x_{\text{true}}$ 로 둔다.

- 각 n 에 대해 $H_n n b$ 를 계산하시오.
- condition number와 error norm을 그래프로 나타내시오.

10. 다중 입력 선형모델 추정

세 변수 x_1, x_2, x_3 와 출력 y 가 다음 관계를 따른다고 하자.

$$y = 1 + 2x_1 - 3x_2 + 0.5x_3 + \epsilon$$

100개의 난수 데이터를 생성하고 least squares로 계수를 추정하시오.

코드답안 1: 고정밀 4×4 선형시스템 분석

```
clear; clc;                                % Clear workspace

A = [8 -2 1 0; ...
     -2 7 -3 1; ...
     1 -3 9 -2; ...
     0 1 -2 6];                             % Coefficient matrix

b = [10; -4; 15; 3];                       % Right-hand side vector

x = A\b;                                    % Solve Ax=b

det_A = det(A);                             % Determinant
rank_A = rank(A);                           % Rank
res_norm = norm(b - A*x);                   % Residual norm

disp(x);
fprintf('det(A) = %.4f\n', det_A);
fprintf('rank(A) = %d\n', rank_A);
fprintf('Residual norm = %.4e\n', res_norm);
```

코드답안 2: 거의 특이한 행렬의 수치 불안정성

```
clear; clc; % Clear workspace

eps_list = [1e-1 1e-3 1e-6 1e-10]; % Epsilon values

for k = 1:length(eps_list)
    eps_val = eps_list(k); % Current epsilon

    A = [1 1; 1 1+eps_val]; % Nearly singular matrix
    b = [2; 2+eps_val]; % Right-hand side

    x = A\b; % Solve system
    c = cond(A); % Condition number

    fprintf('epsilon = %.1e\n', eps_val);
    disp(x);
    fprintf('cond(A) = %.4e\n\n', c);
end
```

코드답안 3: 세 개의 데이터 센서를 이용한 선형 보정 모델

```
clear; clc; close all;           % Clear workspace

rng(1);                          % Fix random seed

x = (0:0.5:8)';                  % Input data
y = 2 + 1.5*x - 0.2*x.^2 + 0.3*randn(size(x)); % Noisy output
A = [ones(size(x)) x x.^2];      % Design matrix
c = A\y;                          % Least squares coefficients

xq = linspace(min(x), max(x), 300)'; % Dense x values
Aq = [ones(size(xq)) xq xq.^2];    % Dense design matrix
y_fit = Aq*c;                     % Fitted curve
plot(x,y,'o','LineWidth',1.5);
hold on;
plot(xq,y_fit,'LineWidth',2);
grid on;
legend('Data','Quadratic fit');
```


코드답안 4: Inconsistent System 판별

```
clear; clc;                                % Clear workspace

A = [1 2 3; 2 4 6; 1 1 1];                % Coefficient matrix
b = [6; 13; 3];                            % Right-hand side vector

rank_A = rank(A);                          % Rank of A
rank_aug = rank([A b]);                    % Rank of augmented matrix
n = size(A,2);                             % Number of unknowns

fprintf('rank(A) = %d\n', rank_A);
fprintf('rank([A b]) = %d\n', rank_aug);

if rank_A < rank_aug
    disp('No solution');
elseif rank_A == n
    disp('Unique solution');
else
    disp('Infinitely many solutions');
end
```

코드답안 5: Overdetermined System에서 Residual 분석

```
clear; clc; close all;           % Clear workspace

t = (0:9)';                      % Input vector
y = [1.2; 2.1; 2.9; 4.2; 5.1; ... % Observed data
     5.8; 7.0; 8.1; 8.9; 10.2];
A = [ones(size(t)) t];          % Design matrix
c = A\y;                         % Least squares solution
y_fit = A*c;                    % Fitted values
r = y - y_fit;                  % Residual vector
res_norm = norm(r);             % Residual norm

disp(c);
disp(r);
fprintf('Residual norm = %.4f\n', res_norm);

plot(t,y,'o',t,y_fit,'-','LineWidth',1.5);
legend('Data','Fitted line');
grid on;
```

코드답안 6: Sparse Tridiagonal System

```
clear; clc; close all;           % Clear workspace

n = 200;                         % Matrix size
e = ones(n,1);                  % Vector of ones

A = spdiags([-e 4*e -e], -1:1, n, n); % Sparse tridiagonal matrix

b = ones(n,1);                  % Right-hand side

x = A\b;                        % Solve system

plot(x,'LineWidth',1.5);
xlabel('Index');
ylabel('Solution value');
title('Sparse Tridiagonal System Solution');
grid on;
```

코드답안 7: 정규방정식과 Backslash 비교

```
clear; clc; % Clear workspace

rng(2); % Fix random seed

x = linspace(-2,2,40)'; % Input data
y = 1 - 2*x + 0.5*x.^2 + ...
    0.8*x.^3 + 0.2*randn(size(x)); % Noisy cubic data

A = [ones(size(x)) x x.^2 x.^3]; % Design matrix

c_backslash = A\y; % Backslash solution
c_normal = inv(A'*A)*(A'*y); % Normal equation solution

disp(c_backslash);
disp(c_normal);
disp(c_backslash - c_normal);
```

코드답안 8: 회로 전류 문제

```
clear; clc;                                % Clear workspace

A = [12 -3 0 -2; ...
     -3 10 -4 0; ...
      0 -4 9 -1; ...
     -2 0 -1 7];                          % Circuit matrix

b = [10; 5; 3; 4];                        % Source vector

i = A\b;                                    % Current vector

[largest_current, idx] = max(abs(i)); % Largest absolute current

disp(i);
fprintf('Largest absolute current = %.4f\n', largest_current);
fprintf('Current index = %d\n', idx);
```

코드답안 9: Hilbert Matrix 실험

```
clear; clc; close all;           % Clear workspace

n_list = [4 6 8 10 12];         % Matrix sizes

cond_values = zeros(size(n_list)); % Preallocate
err_values = zeros(size(n_list)); % Preallocate

for k = 1:length(n_list)
    n = n_list(k);               % Current size
    H = hilb(n);                 % Hilbert matrix
    x_true = ones(n,1);          % True solution
    b = H*x_true;                % Right-hand side

    x_est = H\b;                 % Computed solution

    cond_values(k) = cond(H);     % Condition number
    err_values(k) = norm(x_est - x_true); % Error norm
end
```

코드답안 9: Hilbert Matrix 실험 그래프

```
figure;  
  
semilogy(n_list, cond_values, 'o-', 'LineWidth', 1.5);  
hold on;  
semilogy(n_list, err_values, 's-', 'LineWidth', 1.5);  
  
xlabel('Matrix size n');  
ylabel('Log scale value');  
legend('Condition number', 'Error norm');  
title('Hilbert Matrix Conditioning');  
grid on;
```

코드답안 10: 다중 입력 선형모델 추정

```
clear; clc;                                % Clear workspace

rng(10);                                    % Fix random seed

n = 100;                                    % Number of samples

x1 = randn(n,1);                            % Input variable 1
x2 = randn(n,1);                            % Input variable 2
x3 = randn(n,1);                            % Input variable 3

eps_noise = 0.2*randn(n,1);                % Noise

y = 1 + 2*x1 - 3*x2 + 0.5*x3 + eps_noise; % Output

A = [ones(n,1) x1 x2 x3];                 % Design matrix

c = A\y;                                    % Estimated coefficients

disp(c);                                    % Display coefficients
```